ASSIGNMENT 1 – TASK 3: Design the Control Logic

Bloom's Level: K4 – Analyse | Marks: 4

Allocation uses first-fit order: scan floors and then slots. Stop immediately when a free compatible slot is found. If none exists, return null. Release validates both indexes, checks that the slot is occupied, clears it and returns true; invalid or already-free positions return false.

Algorithm A – allocate(vehicleNo)

```
BEGIN
  IF vehicleNo is null or blank, RETURN null
  FOR each floor f
    FOR each slot s on floor f
      IF slot[f][s] is free
        slot[f][s] = vehicleNo
        RETURN [f, s]     // stop at first match
      END IF
    END FOR
  END FOR
  RETURN null             // no slot available
END
```

Algorithm B – release(floor, slot)

```
BEGIN
  IF floor < 0 OR floor >= numberOfFloors, RETURN false
  IF slot < 0 OR slot >= slots[floor].length, RETURN false
  IF slots[floor][slot] is already free, RETURN false
  slots[floor][slot] = free
  RETURN true
END
```

Algorithm C – fee calculation

```
BEGIN
  IF entryTime is missing OR exitTime is missing, RETURN error
  IF exitTime < entryTime, STOP with "invalid time range"
  minutes = elapsed whole minutes between entryTime and exitTime
  IF eligible student/staff pass is valid at entry
    parkingFee = 0
  ELSE IF minutes = 0
    parkingFee = 0
  ELSE IF minutes <= 60
    parkingFee = first-hour slab
  ELSE IF minutes <= 180
    parkingFee = first-hour slab + next two-hour slab
  ELSE
    parkingFee = first-hour slab + next two-hour slab
                 + each additional started-hour slab
  END IF
  IF dailyMaximum is configured, parkingFee = MIN(parkingFee, dailyMaximum)
  IF EV charging was used
    chargingFee = units * chargingRate
  ELSE
    chargingFee = 0
  END IF
  RETURN parkingFee + chargingFee
END
```

The slabs above define the algorithm structure, not official prices. The campus must set the actual amount for the first hour, the next two hours and each additional started hour. This avoids inventing rates while making the duration rule explicit.

For a fully worked sample tariff, assume ₹20 for the first started hour, ₹10 for every additional started hour, and a daily maximum of ₹100. Thus 0 minutes = ₹0, 1–60 minutes = ₹20, 61–120 minutes = ₹30, 121–180 minutes = ₹40, and any further started hours add ₹10 until the ₹100 daily cap. These are demonstration values only; replace them with the campus-approved tariff.

Working Java code for the supplied String[][] template

```
public class ParkingFacility {
    // slots[floor][slot] holds a vehicle number, or null if the slot is free
    private final String[][] slots;

    public ParkingFacility(int[] slotsPerFloor) {
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            if (slotsPerFloor[f] < 0) {
                throw new IllegalArgumentException("Negative slot count");
            }
            slots[f] = new String[slotsPerFloor[f]];
        }
    }

    /** Returns {floor, slot} of the first free slot, or null if it is full. */
    public int[] allocate(String vehicleNo) {
        if (vehicleNo == null || vehicleNo.trim().isEmpty()) {
            return null;
        }
        for (int f = 0; f < slots.length; f++) {
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) {
                    slots[f][s] = vehicleNo;
                    return new int[] { f, s };
                }
            }
        }
        return null;
    }

    /** Frees a slot. Returns false if the position is invalid or already free. */
    public boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length) return false;
        if (slot < 0 || slot >= slots[floor].length) return false;
        if (slots[floor][slot] == null) return false;
        slots[floor][slot] = null;
        return true;
    }

    public static void main(String[] args) {
        // floor 0 has 2 slots, floor 1 has 1 slot
        ParkingFacility p = new ParkingFacility(new int[] {2, 1});
        System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " " + p.release(5, 0));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

Expected output

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

Alternative concept and complexity

A free-slot stack stores each available (floor, slot) position. If the system also maintains a free count for each floor, it can skip floors whose count is zero. Allocation and release can be O(1) average when a compatible slot is available at the top; a scan may still be needed if slot types differ. A stack uses O(N) space for N slots. A plain nested scan takes O(N) worst-case allocation time and O(1) extra search space. Use break to stop a loop when a matching slot is found; use continue to skip an unavailable/incompatible slot without doing extra work. Do not use break/continue where return clearly expresses the operation's end.

Free-slot stack with per-floor free counts

For N total slots, keep a stack of free positions (floor, slot) and an integer freeCount[f] for each floor. Push each slot onto the stack at initialisation and increment that floor's count. When a slot is allocated, pop a compatible position and decrement its floor count. When a vehicle exits, push the released position back and increment the count. If vehicle/slot compatibility differs, temporarily skip incompatible stack entries and restore them; the worst case can still scan O(N) positions.

```
// Pseudocode: initialise (push in reverse so the first pop gives [0, 0], as in first-fit)
for each floor f from last to first:
    freeCount[f] = slotsPerFloor[f]
    for each slot s on floor f from last to first:
        freeSlots.push(f, s)

// Allocate when any free slot is acceptable
while freeSlots is not empty:
    position = freeSlots.pop()
    if isCompatible(vehicle, position):
        freeCount[position.floor]--
        return position
    else:
        skipped.push(position)   // continue: try another slot

while skipped is not empty:
    freeSlots.push(skipped.pop())
return null

// Release a validated occupied position
freeSlots.push(floor, slot)
freeCount[floor]++
```

If the system guarantees that every vehicle can use every slot, stack allocation and release are O(1) time. With compatibility constraints, finding a suitable stack entry can be O(N) in the worst case. The stack plus the slot array uses O(N) space, while per-floor counts use O(F) additional space for F floors.